



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Curso de Engenharia de Software

Roteiro de Laboratório — Threads em Java

LabDAMD · Unidade 0 · Revisão de SO e Concorrência

2º Semestre de 2026

T1: Prof. Cleiton Silva Tavares • T2: Prof. Cristiano de Macedo Neto

Objetivos do Laboratório

Ao final deste laboratório, você deverá ser capaz de:

01

Processo x Thread

Explicar a diferença entre processo e thread do ponto de vista do SO.

02

Forma clássica

Implementar threads em Java com Thread e Runnable.

03

Forma atual

Implementar a mesma solução com ExecutorService e Virtual Threads (Java 21+).

04

Custo real

Observar na prática o custo de criar milhares de threads de SO.

O problema usado é propositalmente simples — o foco é o mecanismo de concorrência, não a lógica de negócio.

Pré-requisitos do Laboratório

Confirme os itens abaixo antes de iniciar a atividade prática.

1

JDK 21 ou superior

Confira com `java -version`. Sem o JDK 21+, a Parte E (Virtual Threads) não compila.

2

Editor de código

VS Code, IntelliJ ou apenas terminal + `javac` — qualquer um serve.

3

Java básico

Sintaxe de classes, métodos e o método `main()` devem ser familiares.

4

Leitura da revisão conceitual

Reveja processo x thread e o ciclo de vida antes de programar (próximos slides).

Passo a Passo: Threads em Java

Guichê de atendimento — problema simples para revisar concorrência

PARTES DO ROTEIRO

Revisão	Processo x Thread + ciclo de vida da thread
Parte A	extends Thread — forma clássica #1
Parte B	implements Runnable — forma clássica #2
Parte C	Muitas threads — o limite do Sistema Operacional
Parte D	ExecutorService — pool de threads (Java 5+)
Parte E	Virtual Threads — norma atual (Java 21+)
Fechamento	Quadro comparativo, exercícios e checklist

OBJETIVO DO ENCONTRO

Revisar processo x thread e o ciclo de vida de uma thread, comparando a forma clássica com a forma atual de concorrência em Java.

MATERIAL DE APOIO

Roteiro de Laboratório: Threads em Java (PDF/MD)

JDK 21+ instalado em cada máquina

Projektor/tela para o Gerenciador de Tarefas (Parte C)

Processo x Thread

O SO cria um processo para cada programa (a JVM, no nosso caso). Toda thread que criamos é mapeada para uma thread nativa do SO.

- **Memória**

Processo tem espaço de endereçamento próprio; thread compartilha a memória do processo pai.

- **Criação**

Criar processo é custoso (nova tabela de páginas, novo PID); criar thread é mais leve.

- **Comunicação**

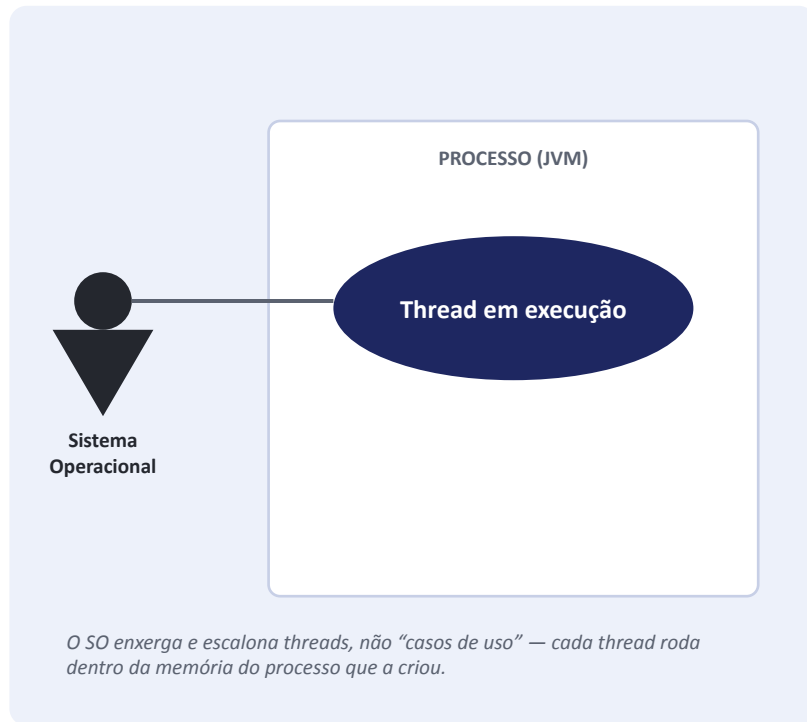
Processos precisam de IPC (pipes, sockets); threads comunicam-se por variáveis compartilhadas.

- **Isolamento**

Falha em um processo não derruba outro; exceção não tratada numa thread pode afetar o processo todo.

- **Escalonamento**

O processo é a unidade de alocação de recursos; a thread é a unidade que a CPU de fato escala.



extends Thread

A forma mais antiga de criar uma thread em Java: a própria classe herda de Thread e sobreescreve run().

- **Herança única**

Uma classe só pode estender uma superclasse — se AtendimentoThread já precisasse herdar de outra classe, essa abordagem não funcionaria.

- **start() x run()**

Chamar start() cria uma nova thread de SO; chamar run() apenas executa o método no thread atual.

- **Nome da thread**

setName() ajuda a identificar cada thread nos logs — essencial para debugar concorrência.

- **join()**

Bloqueia quem chamou até a thread terminar — sem ele, o programa pode encerrar antes dos atendimentos.

- **Pergunta**

O tempo total ficou perto de 1s ou de 5s com 5 atendimentos? Por quê?

```
public class AtendimentoThread
    extends Thread {

    int idCliente;

    public void run() {
        System.out.println(
            getName() +
            " atendendo cliente " +
            idCliente);
        Thread.sleep(1000);
    }
}

var t = new AtendimentoThread();
t.start();    // nunca t.run()
t.join();
```

Se AtendimentoThread precisasse herdar de Funcionario, esta abordagem já não funcionaria mais.

implements Runnable

Separa a tarefa (o que executar) de quem executa (a Thread) — resolve a limitação da herança única.

- **Sem herança presa**

Uma classe pode implementar Runnable e ainda herdar de outra classe qualquer.

- **Reuso da tarefa**

O mesmo Runnable pode ser passado para várias Threads diferentes.

- **new Thread(tarefa)**

A Thread apenas executa a tarefa — ela não é a tarefa, como no modelo anterior.

- **Nome customizado**

`new Thread(tarefa, "Atendente-" + i)` nomeia a thread no momento da criação.

- **Pergunta**

Qual das duas classes (Parte A ou B) você poderia fazer herdar de outra classe hoje?

```
public class AtendimentoRunnable
    implements Runnable {

    int idCliente;

    public void run() {
        System.out.println(
            "atendendo cliente " +
            idCliente);
        Thread.sleep(1000);
    }
}

Runnable r =
    new AtendimentoRunnable();
new Thread(r, "Atendente-1")
    .start();
```

Desde o Java 5, implementar Runnable é a forma recomendada sobre estender Thread diretamente.

Muitas threads de Sistema Operacional

Vamos forçar a criação de 10 mil threads nativas para sentir, na prática, o custo real do modelo clássico.

● Cada thread custa memória

Uma thread nativa reserva tipicamente 512KB–1MB de pilha, além de estruturas do kernel.

● Observando o SO

ps -L (Linux) ou o Gerenciador de Tarefas (Windows) mostram quantas threads o processo criou.

● O limite aparece

Acima de ~50 mil threads, muitas configurações lançam `OutOfMemoryError: unable to create new native thread`.

● Pergunta

Por que criar uma thread de SO é mais caro do que criar um objeto comum em Java?

● Pergunta

O que esse limite sugere sobre usar 1 thread por requisição em um servidor web?

```
int total = 10_000;
Thread[] threads =
    new Thread[total];

for (int i=0; i<total; i++) {
    threads[i] = new Thread() -> {
        Thread.sleep(1000);
    });
    threads[i].start();
}
for (var t : threads) t.join();

// acima de ~50_000:
// OutOfMemoryError:
// unable to create new
// native thread
```

Esse é exatamente o problema que motivou a evolução do modelo de concorrência em Java.

ExecutorService (pool de threads)

Em vez de criar uma thread de SO por tarefa, um pool reaproveita um número fixo de threads reais.

- **Você submete, não cria**

`pool.submit(tarefa)` — quem decide como e quando executar é o próprio pool.

- **Poucas threads reais**

`newFixedThreadPool(4)` atende N tarefas com apenas 4 threads de SO reutilizadas.

- **shutdown()**

Sem ele, o pool continua vivo esperando novas tarefas e o programa nunca termina.

- **Padrão desde 2004**

Essa é a forma recomendada de gerenciar concorrência em Java desde a versão 5.

- **Pergunta**

Com 4 threads atendendo 10 clientes, o tempo total ficou perto de 1s, 2s ou 3s?

```
ExecutorService pool =  
    Executors  
        .newFixedThreadPool(4);  
  
for (int i=0; i<10; i++) {  
    int idCliente = i;  
    pool.submit(() -> {  
        System.out.println(  
            Thread.currentThread()  
                .getName());  
        Thread.sleep(1000);  
    });  
}  
  
pool.shutdown();  
pool.awaitTermination(  
    1, TimeUnit.MINUTES);
```

pool-1-thread-1, pool-1-thread-2... repare que os nomes se repetem entre as tarefas.

Virtual Threads (Java 21+)

Gerenciadas pela própria JVM, não pelo SO — é possível criar milhões sem esgotar a memória do processo.

- **Leveza extrema**

Cada Virtual Thread ocupa uma fração da memória de uma thread de SO tradicional.

- **`newVirtualThreadPerTaskExecutor()`**

Cria uma nova Virtual Thread para cada tarefa — sem medo de esgotar recursos.

- **Carrier threads**

Por baixo, poucas threads de SO reais executam o código quando ele realmente precisa rodar.

- **100 mil sem erro**

O mesmo experimento da Parte C, que quebrava com 50 mil, roda tranquilamente aqui.

- **Pergunta**

Uma Virtual Thread é uma thread de Sistema Operacional? Se não, o que ela é?

```
int total = 100_000;

try (var executor =
    Executors
      .newVirtualThreadPer
        TaskExecutor()) {

    for (int i=0; i<total; i++){
        executor.submit(() -> {
            Thread.sleep(1000);
        });
    }
} // aguarda terminar

// 100.000 atendimentos,
// sem OutOfMemoryError
```

Antes: 1 thread Java = 1 thread de SO. Agora: milhões de Virtual Threads são possíveis.

Ciclo de Vida de uma Thread

Toda thread passa por estados bem definidos entre a criação e o encerramento — entender isso explica por que um programa às vezes “trava” esperando algo.

NEW

Objeto Thread criado, mas `start()` ainda não foi chamado.

RUNNABLE

Pronta para executar ou executando — o escalonador do SO decide.

BLOCKED

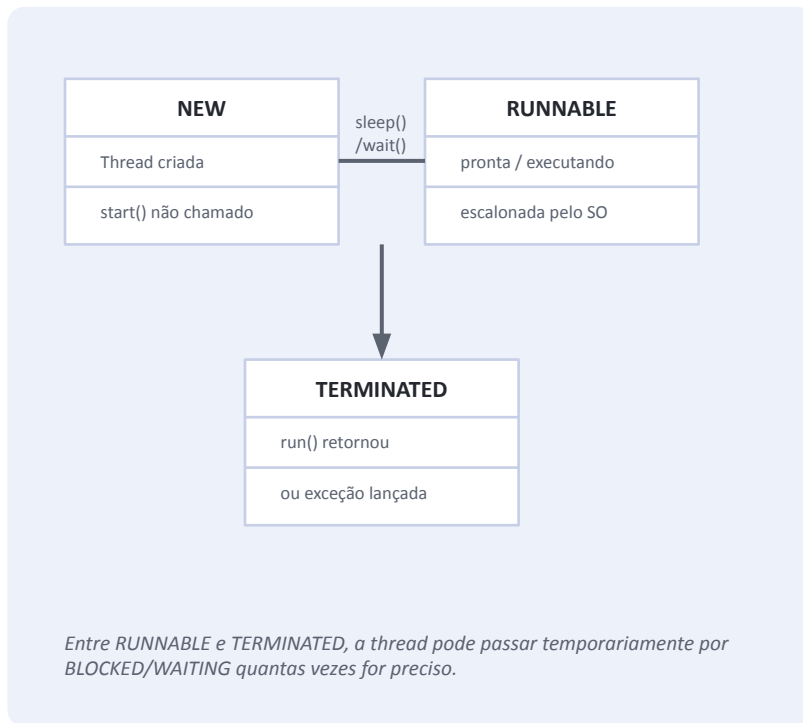
Aguardando a liberação de um lock (monitor) para continuar.

WAITING / TIMED_WAITING

Aguardando `sleep()`, `join()` ou uma notificação explícita.

TERMINATED

O método `run()` retornou ou lançou uma exceção não tratada.



Evolução da Concorrência em Java

Cada geração resolveu o problema de escala da anterior, sem descartar os conceitos de SO revisados nesta aula.

LINHA DO TEMPO

De baixo para cima: da forma mais antiga (Java 1.0) até a norma atual (Java 21+).

Virtual Threads

Java 21+: milhões de threads leves, gerenciadas pela JVM (norma atual).

ExecutorService

Java 5+: pool de threads reais, reaproveitadas para várias tarefas.

Runnable + Thread

Desde sempre: separa a tarefa (Runnable) de quem a executa (Thread).

extends Thread

Java 1.0: 1 thread Java = 1 thread do SO (forma mais antiga).

Virtual Threads

ExecutorService

Runnable + Thread

extends Thread

PARA ENCERRAR

Exercícios e Checklist de Entrega

Atividades para consolidar o conteúdo revisto nesta aula

EXERCÍCIOS DE FIXAÇÃO

- Trocar o pool fixo por `newCachedThreadPool()` — o comportamento muda?
- Imprimir `Thread.currentThread()` na Parte E — é uma `VirtualThread`?
- Explicar processo x thread com a analogia do guichê de atendimento
- Escolher, com justificativa, a abordagem ideal para um servidor com milhares de conexões

CHECKLIST ANTES DE ENTREGAR

- As 5 partes (A a E) compilam e executam sem erro
- As perguntas de cada parte estão respondidas no relatório
- Você consegue explicar processo x thread sem consultar o roteiro

Dúvidas? Procure o professor durante o laboratório.